# CSS

## Block Positioning

floating blocks, position

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

**doc. Ing. Radek Burget, Ph.D.**

burgetr@vut.cz

**Ing. Jiří Hynek, Ph.D.**

hynek@vut.cz

# Layout Modes

- Normal flow
  - So called in-flow elements
  - Elements are laid out in the document order
  - Inline elements on the lines, block elements below each other
- Floating blocks
  - Elements moved to left or right side
  - The `float` property
- Positioned elements
  - The element position is specified explicitly
  - The `position` property

# Floating blocks

- Object that are flowed by the surrounding text
- Creating a floating object:
  - `float: left` or
  - `float: right`
- Floating elements become automaticaly **block** elements
- The floating object is
  - Removed from the normal text flow (doesn't influence the text on the original place)
  - Shifted to the border of the parent element
- Different way of width computation
  - It is better to set an explicit width
  - If not set, it is automaticaly computed (shring-to-fit algorithm)

# Creating a Floating Object

# Creating a Floating Object – Width

# Creating a Floating Object – Width

# Float and In-Flow Block Size

# Clear

- The floating objects are placed by the browser
  - Side by side to the appropriate margin
  - When there is no space then below each other
  - => The floating object can horizontally overflow the flowing text
- When some text shouldn't flow along the floating objects:
  - `clear: left | right | both`
- First, previous floating object are placed, then the text is placed below

```
Float 1Float 2This paragraph has no clear — wraps around the floats.This
                                                                    paragr
                                                                    has
                                                                    clear: b
```

# Clear property

# Clear property

# Containing Block Height

- When a block has `height:auto`, its height depends on its contents
- For `overflow: visible`
  - Only in-flow content is considered
  - If the block contains floating blocks only, it has **zero height** in the result
- Otherwise (e.g. `overflow: hidden`)
  - Floating content is considered too

**overflow: visible** — blue border collapses to zero height:

Column 1
Column 2

**overflow:**

# Positioning

- Manual specification of the object's position on the page
- Relative
  - Moving the object relatively to its normal position
- Absolute
  - Placing an object to a particular place independently on the remaining text
- The `position` property
  - `position: static` – normal flow
  - `position: relative` – relative positioning
  - `position: absolute` – absolute positioning
  - `position: fixed` – fixed positioning
- Affects both the **element iself** and its **child elements**!

# Relative positioning

- The position relative to normal placement
  - `position: relative`
- The block remains in normal flow; it's just moved
- The distance from the top left corner
  - `top: length` - moving down
  - `left: length` - moving right
- The distance from the bottom right corner:
  - `bottom: length` - moving up
  - `right: length` - moving left
- May be combined
- Negative value - reverse direction

# Relative positioning - example

```
This is bold text.
```

```
<style>
    .word {
        font-weight: bold;
    }
</style>
…
This is <span class="word">bold</span> text.
```

# Relative positioning - example

```
This is bold text.
```

```
<style>
    .word {
        font-weight: bold;
        position: relative;
        top: 0.5em;
        left: 20px;
    }
</style>
…
This is <span class="word">bold</span> text.
```

# Absolute positioning

- `position: absolute`
- The object is **removed** from the normal flow
  - Doesn't influence the text in the original position
- It is placed to a particular poisition on the page
  - It can overlap the content being already on that place
  - It might be necessary to „make space"
- The distance from the top left corner of the **containing block**
  - `top: length`
  - `left: length`
- The distance from the bottom left corner
  - `bottom: length`
  - `right: length`

# Absolute positioning - example

```
This is bold text.
```

```
<style>
    .word {
        font-weight: bold;
        position: absolute;
        bottom: 0em;
        left: 20px;
    }
</style>
…
This is <span class="word">bold</span> text.
```

# Containing Block

- Initial containing block is the Viewport
- Elements with position `relative` or `static`
  - The nearest ancestor block is the containing block
- Elements with position `absolute`
  - The nearest ancestor block with position `relative`, `absolute` or `fixed` is the containing block
- Elements with position `fixed`
  - Viewport is always the containing block

# Coordinate System

- Normally, the coordinate system is formed by the **Viewport**
- Elements that have `position: absolute`, `relative` or `fixed`, create a **local coordinate system** for absolute positioning
  - Nested object with `position: absolute` are positioned from within the parent

# Coordinate System (II)

child1 (absolute)

- Child blocks are positioned within **Viewport** (parent is `static`):

Parent (position: static)

```
<div id="parent">
    Parent (position: static)
    <div id="child1" style="position: absolute">…</div>
    <div id="child2" style="position: absolute">…</div>
</div>
```

child2 (absolute)

# Coordinate System (III)

- Child blocks are positioned within **parent block** (parent is `relative`):

Parent (position: relative)

child1 (absolute)

child2 (absolute)

```
<div id="parent" style="position: relative">
    Parent (position: relative)
    <div id="child1" style="position: absolute">…</div>
    <div id="child2" style="position: absolute">…</div>
</div>
```

# Fixed positioning

```
<style>
    .word {
        font-weight: bold;
        position: fixed;
        top: 150px;
        left: 500px;
    }
</style>
…
This is <span class="word">bold</span> t
```

Normal text content here. The bold word

# Positioned Element Width and Height

- The equation for width: **left** + margin-left + border-left-width + + padding-left + width + padding-right + + border-right-width + margin-right + **right** = containing_block_width
- The `auto` values for `left` and `top` are derived from static position
- Computation of multiple `auto` values is defined in the specification
- The **height** is computed analogously

# Element overlaping

- Two elements may overlap (relative or absolute positioning)
- Normally, the later specified object overlaps the earlier object
- This can be changed using the `z-index` property
  - `z-index: 10`
  - The number gives the Z coordinate of the object (the higher number the „higher" placed is the object)
  - If not specified, **0** is used

# Overlaping: example

First box
Second box

```
<div style="height: 4em; position: relative">
    <div class="box" style="position: absolute;
         top: 30px; left: 180px">
         First box</div>
    <div class="box" style="position: absolute;
         top: 60px; left: 200px">
         Second box</div>
</div>
```

# Overlaping: example

First box
Second box

```
<div style="height: 4em; position: relative">
    <div class="box" style="position: absolute;
         top: 30px; left: 180px; z-index: 1">
         First box</div>
    <div class="box" style="position: absolute;
         top: 60px; left: 200px">
         Second box</div>
</div>
```

# Object hiding

- Two possibilities
  1. `display: none`
     - The element is totally ignored during rendering
  2. `visibility: hidden`
     - The object occupies the place on the page but it is not displayed

# Visibility – example

```
This is <span style="…">invisible</span> text.
```

1. `display: none`

This is invisible text. (gap removed)

2. `visibility: hidden`

This is  text. (space preserved)

# Why to hide the text

1. `display: none`
   - The elements displayed in browsers with no CSS support (alternative navigation, auxiliary labels, …)
   - Element intended for other media types (e.g. for printing)
2. `visibility: hidden`
   - The elements that may be displayed later (scripting)

# Table style

- We can change the style of
  - The whole table - `<table>`
  - Rows - `<tr>`
  - Cells - `<td>`, `<th>`
  - Columns - `<col>`, `<colgroup>`
  - Header, footer, body - `<thead>`, `<tfoot>`, `<tbody>`
  - Caption - `<caption>`

# Table caption

Tab 1. Caption on the top

| Name | Surname |
|---|---|
| John | Wayne |
| Albert | Einstein |

Tab 2. Caption in the bottom

| Name | Surname |
|---|---|
| John | Wayne |
| Albert | Einstein |

```
.top caption { caption-side: top; }
.bottom caption { caption-side: bottom; }
```

# Borders in a table

- Borders can be set separately for the cells or the whole table
- Special properties:
  - `border-collapse: separate | collapse` - cell border merging
  - `border-spacing: 5px` - cell border spacing
  - `empty-cells: show | hide` - whether to diplay or not the empty cells

# Border Examples

`border-collapse: separate`

| Name | Surname |
|---|---|
| John | Wayne |
| Albert | Einstein |

`border-collapse: collapse`

| Name | Surname |
|---|---|
| John | Wayne |
| Albert | Einstei |

```
/* separate borders (default) */
table { border-collapse: separate; }
/* merged borders */
table { border-collapse: collapse; }
/* horizontal lines only */
table tr { border-top: 1px solid; }
/* full outer + left cell borders */
table { border: 1px solid; }
table th { border: 1px solid; }
table td { border: none; border-left: 1px solid; }
```

# Lists: Bullet and Numbering Types

| Unordered (`ul`) | Ordered (`ol`) |
|---|---|

- The `l` … o th

```
ul li { list-style-type: square; }
ol li { list-style-type: lower-roman; }
```

| | |
|---|---|
| ◦ circle | i. lower-roman |
| ▪ square | I. upper-roman |

# Bullet position

- The `list-style-position` property

```
list-style-position: outside
```

- First item in longer text in longer text in longer

(default):

# Mouse cursor

- The `cursor` property allows to change the mouse cursor look when placed over the particular element on the page.

```
The cursor over thisword
                        has a different shape.
```

```
The cursor over this
<span style="cursor: crosshair">word</span>
has a different shape.
```

# Cursor types

| | | |
|---|---|---|
| auto | crosshair | default |
| hand | pointer | move |
| text | wait | help |
| nw-resize | n-resize | ne-resize |
| w-resize | | e-resize |
| sw-resize | s-resize | se-resize |

# Pseudo Elements and Classes

- Abstract class for denoting special elements or their state
- They define the look of various parts
  - **Pseudo classes** : in different states
    - E.g. links visited, not visited, focused, etc.
    - Specificity as for classes
    - Denoted by „:"
  - **Pseudo elements** : different parts of elements
    - First element, first letter, before, after, …
    - Specificity as for elements
    - Denoted by „::"
  - `p::first-letter { font-size: 20px; }`

# Pseudo Classes: Link Style

- Static
  - Not visited link: `a:link { color: green; }`
  - Visited link: `a:visited { color: red; }`
- Dynamic
  - Mouse moved over: `a:hover { … }`
  - Link activation: `a:active { … }`
  - Keyboar focus: `a:focus { … }`

# Typical Link Style

- Highlighted by color, underlines on moving mouse over
- It gets highlighed when activated, gets darker when visited

```
Example: Sample link.
```

```
a { color: #aaf; text-decoration: none; }
a:link { }
a:visited { color: #339; }
a:hover { text-decoration: underline; }
a:active { text-decoration: underline; color: red; }
```

# Pseudo Class :first-child

- The first child element of its parent element

```
• Blue item

• Green item

• Green item
```

```
<ul class="menu">
    <li>Blue item</li>
    <li>Green item</li>
    <li>Green item</li>
</ul>
```

# Pseudo Elements: Firtst Letters and Lines

- Element `::first-line`
  - Denotes the first line of an arbitrary element
  - E.g. `p::first-line` - first line of a paragraph
  - Depends on the window width, font size, …
- Element `::first-letter`
  - Denotes first letter of an element
  - E.g. `p::first-letter` - first letter of a paragraph

# Example 1

First line of this paragraph will be in capital letters. Any text follows.

```
p { text-transform: none; }
p::first-line { text-transform: uppercase; }
```

```
<p>First line of this line will
be in capital letters. Any text follows. Any text follows.
Any text follows.</p>
```

# Example 2

First letter will be large and floated by the remaining text. Other text a

```
p::first-letter {
    font-size: 200%;
    font-weight: bold;
    float: left;
}
```

```
<p>First letter will be large and floated
by the remaining text. Other text as normal. Other text as normal.
Other text as normal.</p>
```

# Generated content

- Allows to generate a text that is not present in the document
- Pseudo-elements `::before` a `::after`
- The `content:` property

```
This rule doesn't always apply.
```

```
div.remark::before {
    content: "Remark: ";
    font-weight: bold;
}
```

```
<div class="remark">This rule doesn't always apply.</div>
```

# To be continued…

CSS – Creating a complex layout